⚙️ **1) Prerequisites (install + verify)**

- **Java (JDK 17)**

Install JDK 17 (you already have it). Verify:

```
java -version
```

✓ Should show 17.x

- **Maven**

Verify:

```
mvn -version
```

✓ Shows Maven + Java 17

- **Docker Desktop**

Start Docker Desktop → wait until Engine running

Verify:

```
docker --version
docker run hello-world
```

- **Jenkins**

Open:

```
http://localhost:8081
```

🔌 **2) Jenkins Plugins (install once)**

Go:

👉 **Manage Jenkins → Plugins → Available**

Install:

Git

Maven Integration

Docker

Docker Pipeline

Credentials Binding

Pipeline

👉 **Restart Jenkins after install**

🔧 **3) Configure Tools (VERY IMPORTANT)**

Go:

👉 **Manage Jenkins → Global Tool Configuration**

✓ JDK

Name: JDK17

JAVA_HOME:

C:\Program Files\Eclipse Adoptium\jdk-17.0.12.7-hotspot

✓ Maven

Name: Maven3

Install automatically ✓ (or give path if manual)

🔐 **4) DockerHub Access Token (NOT password)**

Go to:

👉 **Docker Hub**

→ Account Settings → Security → Generate Token

Copy token (only shown once)

🔐 **5) Add Credentials in Jenkins**

Go:

👉 **Manage Jenkins → Manage Credentials → Global → Add**

Fill:

Kind: Username with password

Username: your-dockerhub-username

Password: <PASTE TOKEN>

ID: dockerhub-creds

📁 **6) Project Files (ALL FILES)**

📄 **App.java**

```
package com.demo;

public class App {
    public static void main(String[] args) throws Exception {
        System.out.println("Java CI/CD Application Started...");
        while (true) {
            Thread.sleep(5000);
            System.out.println("Hello from Java CI/CD Pipeline!");
        }
    }
}
```

📄 **AppTest.java**

```
package com.demo;
```

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class AppTest {
    @Test
    public void testApp() {
        assertTrue(true);
    }
}
```

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.demo</groupId>
  <artifactId>simple-java-app</artifactId>
  <version>1.0</version>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.10.1</version>
        <configuration>
          <source>17</source>
```

```
        <target>17</target>
      </configuration>
    </plugin>
  </plugins>
 </build>
</project>
```

📄 Dockerfile

```
FROM eclipse-temurin:17-jdk
WORKDIR /app
COPY target/simple-java-app-1.0.jar app.jar
CMD ["java", "-jar", "app.jar"]
```

📄 deployment.yaml (optional)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: java-app
  template:
    metadata:
      labels:
        app: java-app
    spec:
      containers:
      - name: java-app
        image: your-dockerhub-username/java-app:latest
```

🌐 7) Push to GitHub

```
git init
git add .
git commit -m "Initial commit"
```

```
git branch -M main
git remote add origin https://github.com/<your-username>/<repo>.git
git push -u origin main
```

⚙️ **8) Jenkins Pipeline (FINAL)**

```
pipeline {
  agent any

  tools {
    maven 'Maven3'
    jdk 'JDK17'
  }

  environment {
    DOCKER_IMAGE = 'your-dockerhub-username/java-app'
    IMAGE_TAG = 'latest'
  }

  stages {

    stage('Clone') {
      steps {
        git 'https://github.com/<your-username>/<repo>.git'
      }
    }

    stage('Build') {
      steps {
        bat 'mvn clean package'
      }
    }

    stage('Test') {
      steps {
```

```
            bat 'mvn test'
        }
    }

    stage('Docker Build') {
        steps {
            bat 'docker build -t %DOCKER_IMAGE%:%IMAGE_TAG% .'
        }
    }

    stage('Docker Push') {
        steps {
            withCredentials([usernamePassword(
                credentialsId: 'dockerhub-creds',
                usernameVariable: 'DOCKER_USER',
                passwordVariable: 'DOCKER_PASS'
            )]) {
                bat '''
                echo %DOCKER_PASS%| docker login -u %DOCKER_USER% --password-stdin
                docker push %DOCKER_IMAGE%:%IMAGE_TAG%
                '''
            }
        }
    }
  }
}
```

▶️ 9) Run Pipeline

👉 Jenkins → Build Now

🔍 10) HOW TO SEE OUTPUT (VERY IMPORTANT)

✓ Jenkins Console

Click build → Console Output

You'll see:

BUILD SUCCESS

docker build

docker push

✓ DockerHub (MAIN OUTPUT)

Go to:

👉 Docker Hub

Check:

Repository: java-app

Tag: latest

"Last pushed" time

✓ CLI Check

docker pull your-dockerhub-username/java-app:latest

✓ Run Container

docker run -d your-dockerhub-username/java-app

docker logs <container_id>

Output:

Hello from Java CI/CD Pipeline!

🧠 FINAL UNDERSTANDING

Component	Role
GitHub	Source code
Jenkins	Automation
Maven	Build + test
Docker	Package app

DockerHub **Store image**

Kubernetes **Deploy (optional)**

🎯 **WHAT YOU CAN SAY (VIVA)**

"I implemented a CI/CD pipeline where Jenkins automates build, testing, Docker image creation, and pushes it to DockerHub using secure credentials."

Pipeline script: (MAVEN PROJECT)

```
pipeline {
  agent any

  tools {
    maven 'Maven3'
    jdk 'JDK17'
  }

  environment {
    DOCKER_IMAGE = 'your-dockerhub-username/java-app'
    IMAGE_TAG = 'latest'
    DOCKER_BUILDKIT = '0'
  }

  stages {

    stage('Clone Code') {
      steps {
        git branch: 'main',
          url: 'https://github.com/your-username/your-repo.git'
      }
    }

    stage('Build with Maven') {
```

```
    steps {
        bat 'mvn clean compile'
    }
}

stage('Run JUnit Tests') {
    steps {
        bat 'mvn test'
    }
    post {
        always {
            junit allowEmptyResults: true, testResults: 'target/surefire-reports/*.xml'
        }
    }
}

stage('Package Application') {
    steps {
        bat 'mvn package -DskipTests'
    }
}

stage('Check Target Folder') {
    steps {
        bat 'dir target'
    }
}

stage('Docker Build') {
    steps {
        bat '''
        set DOCKER_BUILDKIT=0
        docker build -t %DOCKER_IMAGE%:%IMAGE_TAG% .
```

```
        '''
    }
}

stage('Docker Push') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: 'dockerhub-creds',
            usernameVariable: 'DOCKER_USER',
            passwordVariable: 'DOCKER_PASS'
        )]) {
            bat '''
            docker logout
            echo %DOCKER_PASS%| docker login -u %DOCKER_USER% --password-stdin
            docker push %DOCKER_IMAGE%:%IMAGE_TAG%
            '''
        }
    }
}

// ============================
// OPTIONAL: Kubernetes Deploy
// ============================
/*
stage('Deploy to Kubernetes') {
    steps {
        withCredentials([file(credentialsId: 'kubeconfig', variable: 'KUBECONFIG')]) {
            bat '''
            set KUBECONFIG=%KUBECONFIG%
            kubectl apply -f deployment.yaml --validate=false
            '''
        }
    }
```

```
        }
        */
    }

    post {
        success {
            echo 'Pipeline executed successfully!'
        }
        failure {
            echo 'Pipeline failed. Check logs.'
        }
    }
}
```